



Синтез и анализ на алгоритми

за факултет

”Компютърни Системи и
Технологии”

проф. д-р Даниела Минковска

каб. 2324, кат. ПКТ, ФКСТ, ТУ-София

daniela@tu-sofia.bg



Лекция 2

Сложност на алгоритъм. Анализ на алгоритъм. Размер на задача. Сложност на алгоритъм - минимална, средна, максимална, асимптотична.



Сложност на алгоритъм

Сложност – метрика, която отразява порядъка на броя операции, необходими за изпълнение на дадена операция или алгоритъм, като функция от входните данни, или оценка на броя стъпки за изпълнение на даден алгоритъм.

Означава се с нотацията **$O(f)$** ,
където **f** – е функция на обема на входните данни.



Сложност на алгоритъм. Видове сложност

Сложността може да бъде следните видове:

1. **Константна $O(1)$** - за изпълнение на дадена операция са необходими константен брой стъпки, и те не зависят от обема на входните данни.

2. **Линейна $O(N)$** – за извършване на дадена операция върху N – елемента са необходими приблизително толкова стъпки, колкото са елементите

Напр: за 1000 ел-та са нужни 1000 стъпки

3. **Квадратична $O(n^2)$** – за извършване на дадена операция са нужни n^2 на брой стъпки, където n характеризира обема на вх. данни.

Напр: за дадена операция върху 100 ел-та са нужни 10 000 стъпки



Сложност на алгоритъм. Видове сложност

4. **Кубична $O(n^3)$** - за извършване на дадена операция са нужни n^3 на брой стъпки, където n характеризира обема на вх. данни.

Напр: за дадена операция върху 100 ел-та са нужни 1 000 000 стъпки

5. **Логаритмична $O(\log(N))$** – за извършване на дадена операция върху N – елемента са необходими брой стъпки приблизително $\log(N)$, където основата на логаритъма е най-често 2.

Напр: за $N=1\,000\,000$ ще се направят приблизително 20 стъпки.

6. **Експоненциална $O(2^n)$, $O(N!)$, $O(n^k)$** , – броят стъпки са в експоненциална зависимост спрямо размера на входните данни.

Напр: при $N=10$ експоненциалната функция 2^n има ст-ст 1024;

при $N=20$ експоненциалната функция 2^n има ст-ст 1048576;

при $N=100$ експ. функция 2^n е число с приблизително 30 цифри.



Производителност. Влияние върху производителността

Производителност (Бързодействие) – на компютърната система отразява скоростта на обработката на данните. Измерва се в **брой операции (инструкции) изпълнени за единица време (sec.)**

Производителността зависи от 2 фактора:

1. Технологията (елементната база);
2. Компютърната архитектура.

Производителността се оценява в следните мерни единици:

MIPS – милиони инструкции за секунда;

MOPS - милиони операции за секунда;

MFLOPS – милиони операции за числа с плаваща запетая за секунда.

Друг (важен) фактор е избраният алгоритъм и възможността за неговата оптимизация



Производителност. Влияние върху производителността

Време за изпълнение – скоростта на изпълнение на програмата е в пряка зависимост от сложността на алгоритъма.

Ако **сложността е малка**, програмата ще **работи бързо** дори и за голям обем от данни;

Ако **сложността е висока**, програмата ще **работи бавно** (ще „заспи“) при голям обем от елементи;



Производителност. Влияние върху производителността

Пример: при средно статистически компютър изпълняващ 50 млн. елементарни операции в sec., следват изводи за скоростта на изпълнение на дад. пр-ма в зависимост от сложността на алгоритъма и обема на входните данни:

Сложност на алгоритъм	10	20	50	100	1 000	10 000	100 000
$O(1)$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec
$O(N)$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec
$O(n^2)$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	2 sec	3-4 min
$O(n^3)$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	20 sec	5,55 часа	231,5 дни
$O(\log(N))$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec
$O(2^n)$	< 1 sec	260 дни	заспива	заспива	заспива	заспива	заспива
$O(N!)$	< 1 sec	заспива	заспива	заспива	заспива	заспива	заспива
$O(n^k)$	3-4 min.	заспива	заспива	заспива	заспива	заспива	заспива

Изводи:

1. $O(1)$, $O(N)$ и $O(\log(N))$ - толкова бързи, че няма забавяне и при много вх.данни;
2. $O(n^2)$ – работи добре до няколко хиляди вх. данни;
3. $O(n^3)$ – работи добре под хиляда вх. данни;
4. Експ. алг-ми не работят добре и трябва да се избягват при данни над 10-20;



Примери за оценка на сложност.

1. Единичен цикъл от 1 до N

(*линейна сложност $O(N)$*)

```
int findMaxEl (int[] array)
{
    int max = array[0];
    for (int i=0; i<array.length; i++ )
    {
        if (array[i] > max)
            max= array[i];
    }
    return max;
}
```

Кодът работи добре дори и при голям брой елементи!

2. Два вложени цикъла от 1 до N

(*квадратична сложност $O(n^2)$*)

```
int FindInversions (int[] array)
{
    int inversions = 0;
    for (int i=0; i<array.length; i++ )
        for (int j=0; j<array.length; j++ )
        {
            if (array[i] > array[j])
                inversions++;
        }
    return inversions;
}
```

Кодът работи добре при брой елементи на масива <1000!



Сложност на алгоритъм. Размер на задачата

Размер на задачата (входен размер) – величина, показваща какво е количеството на входните данни на даден алгоритъм – означава се с **n** . Обикновено е = точно на броя на входните данни. т.е. **n** е голямо при голям обем входни данни.

Пр.1: при алгоритми за операции с квадратни матрици $n \times n$, размерът е n , въпреки, че броят елементи е n^2 ;

Пр.2: при алгоритми за графи - n е броят на възлите, или на клоните или сума от броя на възлите;

Пр.3: при алгоритми за електронни схеми - n е броят на транзисторите в схемата.



Сложност на алгоритъм. Времева сложност

Времето за изпълнение на даден алгоритъм T може да се представи като функция на размера на задачата n , т.е. $T(n)$ се нарича **времева сложност**

(аналогично е за сложността по памет - $P(n)$)

Конкретната $T(n)$ зависи от конкретният компютър на който се изпълнява задачата (алгоритъма). $T(n)$ може да се изрази и чрез броя на операциите (които той извършва при изпълнението си) като функция на n .

На практика се отчитат само 1 или 2 основни операции, които са най-трудоемки и се изпълняват най-голям брой пъти в алгоритъма.

Определянето на сложността $T(n)$ се нарича анализ на алгоритъма!

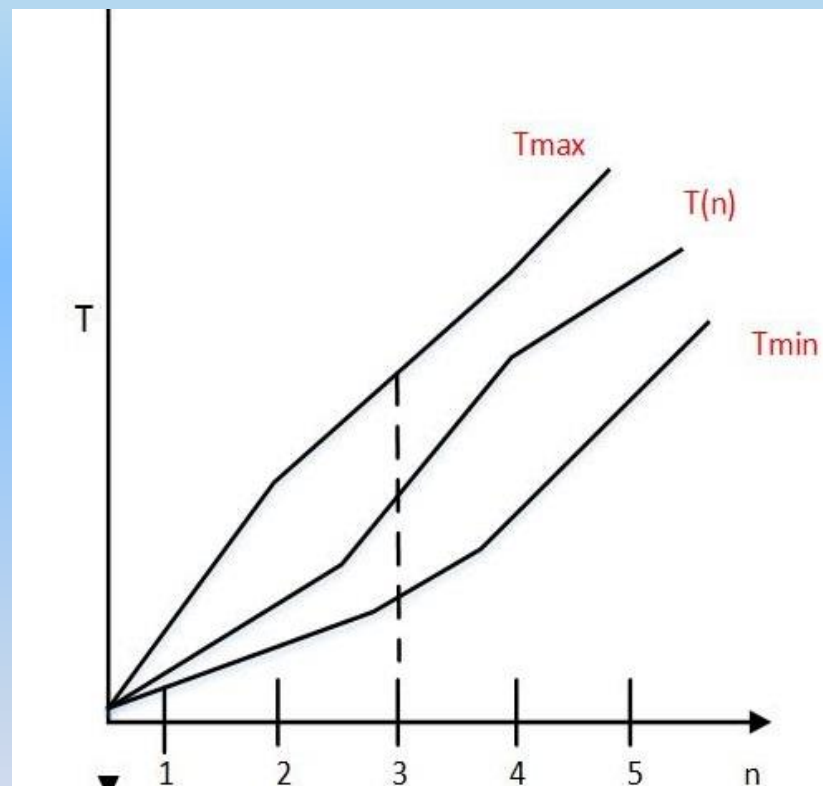
Сложност на алгоритъм. Времева сложност

При един и същи компютър и едно и също n (размер на задачата), в зависимост от конкретните входни данни има:

T_{min} - минимална времева сложност

T_{max} - максимална времева сложност.

Напр. При алгоритъм за задача с квадратна матрица, при зададено n , времето T зависи от това, кои клетки са $\neq 0$, т.е. от запълването на матрицата със стойности, $\Rightarrow$ определянето на T_{max} е най-важно!



Сложност на алгоритъм. Времева сложност

Важно е и определянето на **средната сложност $Top(n)$** – това е средното време, от времената за всички случаи при дадено **n** . Обикновено за определяне на **$Top(n)$** се налага вероятностен анализ. (*затова е труден анализа*)

Точната функция за сложност на алгоритъм се определя от броя на операциите, извършвани при неговото изпълнение!

Напр. За всеки блок – **B_i** (или оператор) се определят 2 величини:

- броят **t_i** на операциите в него, и броят **n_i** на неговите изпълнения,

$\Rightarrow$ **сложността на блока е: $B_i = t_i n_i$**

Ако знаем t_i и n_i на всички блокове $\Rightarrow$ **общата сложност на алг. е:**

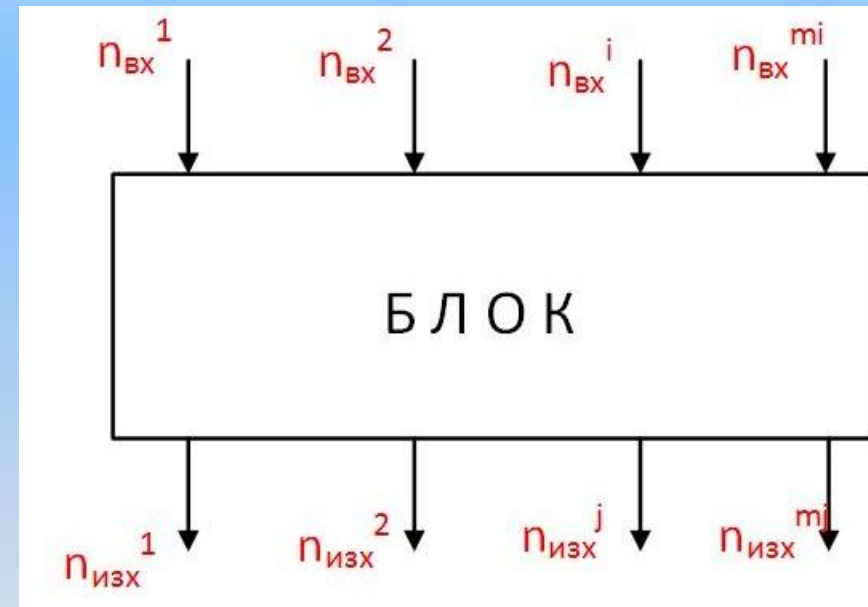
$T(n) = \sum_{i=1}^m n_i t_i$,където **$m$** е броят на блоковете в алгритъма.

Сложност на алгоритъм. Времева сложност

При анализ на алгоритми важи следният **закон**:

Сумата от броя на изпълненията на входните стрелки в даден блок е равен на сумата от броя на изпълнение на изходните стрелки от този блок:

$$\sum_{i=1}^m n_{\text{ВХ}}^i = \sum_{j=1}^n n_{\text{ИЗХ}}^j$$



Сложност на алгоритъм. Времева сложност

Най-лесно и най-оптимално е да се определи функция, която да е някаква граница (най-често горна) на $T(n)$ – това е т.нар. **асимптотична сложност**. При нея се отчитат най-важните членове в израза на $T(n)$, а другите се пренебрегват. Тя оценява сложността на алгоритъма за големи стойности на n ($n \rightarrow \infty$). При нея е важна скоростта на нарастване на функцията.

Напр. В израза $T(n) = a_0 + a_1n + a_2n^2 + a_3n^3$ се оставя само n^3 , другите членове се пренебрегват, и $\implies T(n)$ е от ред n^3

Сложност на алгоритъм. Времева сложност

Видове асимптотична сложност:

1. $T(n) = O(f(n))$, ако съществуват положителни константи $C > 0$; $n_0 > 0$, такива, че $T(n) \leq C \cdot f(n)$ за $n \geq n_0$, т.е. $T(n)$ расте по-бавно или със скорост равна на $f(n)$, $\implies f(n)$ е горна асимптотична граница на $T(n)$;
2. $T(n) = \Omega(f(n))$, ако съществуват положителни константи $C > 0$; $n_0 > 0$, такива, че $T(n) \geq C \cdot f(n)$, т.е. $T(n)$ расте по-бързо или със скорост равна на $f(n)$, $\implies f(n)$ е долна асимптотична граница на $T(n)$;
3. $T(n) = \theta(f(n))$, ако $T(n) = O(f(n))$, и $T(n) = \Omega(f(n))$, т.е. $T(n)$ и $f(n)$ растат с една и съща скорост;
4. $T(n) = o(f(n))$, ако $T(n) = O(f(n))$, и $T(n) \neq \theta(f(n))$, т.е. $T(n)$ расте



Сложност на алгоритъм. Времева сложност

Примери:

1. Нека $T(n) = 500n$ и $f(n) = n^2 \implies T(n) = O(n^2)$, т.к. $500n \leq c \cdot n^2$ за $500 \leq c \cdot n_0$

Напр: При $c = 2$ и $n_0 = 250$ или $c = 4$ и $n_0 = 125 \implies n^2$ е горна граница за $500n$. По – близка граница за $500n$ е:

$g(n) = n$ и $\implies T(n) = O(n) = \Omega(n) = \theta(n)$.

2. При $T(n) = 3n^2 + 2n + 1$ и $f(n) = n^2$, имаме:

$T(n) = O(n^2) = \Omega(n^2) = \theta(n)$, тук се пренебрегват константите в израза, т.к. при големи стойности на n те не влияят!



Сложност на алгоритъм. Времева сложност

Известни са следните правила:

1. Ако $T_1 = O(f(n))$ и $T_2 = O(g(n))$ тогава:

- $T_1 + T_2 = \max(O(f(n)), O(g(n)))$

- $T_1 * T_2 = (O(f(n)) * O(g(n)))$

2. Ако $T(n)$ е полином от степен m , $\Rightarrow T(n) = \theta(n^m)$

3. $\log_k n = O(n)$ за коя да е константа k . $\Rightarrow$ логаритмите растат много по-бавно от линейната функция.



Сложност на алгоритъм. Общи правила

Общи **правила** при определяне сложността на алгоритъма:

1. При цикъл `for`, сложността е = на броя на изпълненията на цикъла, умножен по сложността на тялото.
2. При вложени цикли `for` общото време е = на времето на тялото на цикъла, умножено по произведението на броевете на изпълнението на всеки цикъл.

Напр.: при 3 вложени цикъла:

`for (i=0; i<=n1; i++)`

`for (j=0; j<=n2; j++)`

`for (k=0; k<=n3; k++)`

Сложността е $n_1 * n_2 * n_3 * t$, където t е време за изпълнение на тялото на вътрешния цикъл



Сложност на алгоритъм. Общи правила

3. При последователно изпълнение на оператори (*напр. съставен $\{ \}$*), времената на всички оператори се сумират, но максималната сумирана сложност се определя от оператора с максимална сложност.

4. При условния оператор *if ОП1 – else ОП2*, сложността се определя от времето за проверка + по-голямото от двете времена на ОП1 и ОП2.



Примери за определяне на сложност на алгоритъм

Пр.1: Алг. за намиране на max елемент в 1-мерен масив $A[1..n]$ от реални елементи

```
const int nmax=1000;
```

```
int i,n;
```

```
float A[nmax], max;
```

```
main ()
```

```
{ max = A[0];
```

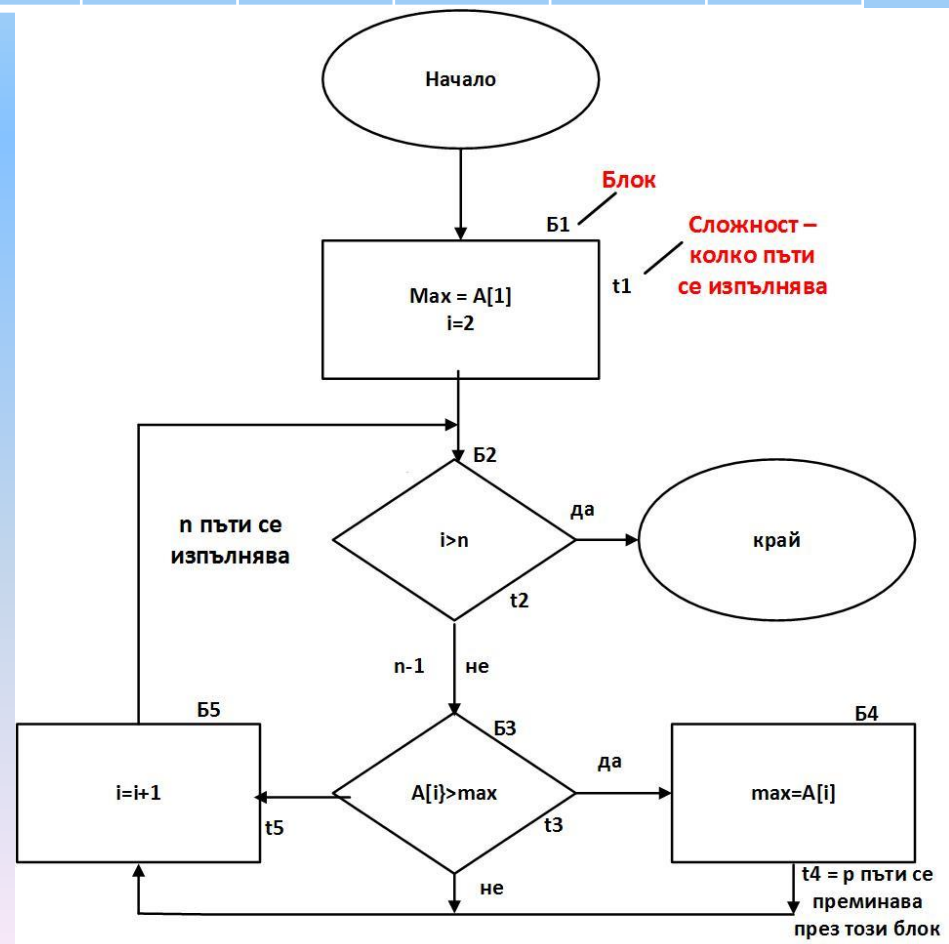
```
  for (i=0;i<n;i++)
```

```
    if (A[i]>max)
```

```
      max=A[i]; }
```

Алг. се обхожда последователно, като се разделя на обходена (с индекси от 1 до $i-1$) и необходена с индекси от i до n части.

23	4	35	7	18	22
----	---	----	---	----	----





Примери за определяне на сложност на алгоритъм

Пр.1: Алг. за намиране на *тах* елемент в 1-мерен масив $A[1..n]$ от реални елементи

Времева сложност: Приемаме, че всеки блок в алг. изисква 1 операция: за Б1 $\rightarrow n_1=1$, и изход Б2 се изпълнява $n-1$ пъти. Блок Б2 се изпълнява n пъти, т.к. $i=2,3,\dots,n,n+1$. Б4 се изпълнява p на брой пъти:

- Ако $A[1]$ е *тах*. $\rightarrow$ Б4 не се изпълнява и $p=0$.
- Ако масива е сортиран възходящо $\rightarrow p_{\max} = n-1$

За определяне на среден брой преминавания $p_{\text{ср}}$, нека приемем, че $n=3$ и няма еднакви елементи (табл.)
 $\Rightarrow p_{\text{ср}(3)} = 5/6$, т.к. за 6 случая ($n!=3!=6$) има 5 преминавания през Б4

Случай	p
$A[1] < a[2] < a[3]$	2
$A[1] < a[3] < a[2]$	1
$A[2] < a[1] < a[3]$	1
$A[2] < a[3] < a[1]$	0
$A[3] < a[1] < a[2]$	1
$A[3] < a[2] < a[1]$	0

В този алг. основната операция е 1: блок Б4 (if), и спрямо нея сложността е: $T(n) = T_{\min}(n) = T_{\max}(n) = T_{\text{ср}}(n) = n-1$ сравнения!



Примери за определяне на сложност на алгоритъм

Пр.2: Алг. за намиране на минималния (*min*) и максималния (*max*) елемент в 1-мерен масив $A[1..n]$ от реални елементи – 3 метода:

- **1 вариант- с 2 обхождания със стъпка 1:**

23	4	18	7	35	22	8	15
За масива $\min=4$, а $\max = 35$							

```
const int nmax=5000;
```

```
int i,n,imin;
```

```
float a[nmax], min,max;
```

```
main ()
```

```
{
```

```
min=a[0];imin=0;
```

```
for (i=1;i<n;i++)
```

```
if (a[i]<min)
```

```
{min=a[i];imin=i;}
```

```
max=a[0];
```

```
for (i=1;i<imin-1;i++)
```

```
if (a[i]>max)
```

```
max=a[i];
```

```
}
```

при 1-то обхождане се определя мин. ел-т (*min*), а при 2-то макс. ел-т (*max*).

За сложността на алг. се отчита само броя на сравненията като основна операция. $\Rightarrow$ за определяне на минимума ще се извършат $n-1$ сравнения, а за максимума $n-2$.

Общата сложност на алг. е $2n-3$

Времевата сложност е:

$$T(n)=T_{\min}(n)=T_{\max}(n)=T_{cp}(n) = n-1$$

Примери за определяне на сложност на алгоритъм

Пр.2: Алг. за намиране на минималния (*min*) и максималния (*max*) елемент в 1-мерен масив $A[1..n]$ от реални елементи – 3 метода:

- **2 вариант- с 1 обхождане със стъпка 1:**

23	4	18	7	35	22	8	15
За масива $\min=4$, а $\max = 35$							

```
const int nmax=5000;
```

```
int i,n,imin;
```

```
float a[nmax], min,max;
```

```
main ()
```

```
{
```

```
min=a[0];max=a[0];
```

```
for (i=1;i<n;i++)
```

```
if (a[i]<min)
```

```
min=a[i]
```

```
else
```

```
if (a[i]>max)
```

```
max=a[i];
```

```
}
```

Min и *max* се търсят едновременно с 1 обхождане.

За алг. броят на сравненията зависи от подредбата на елементите в масива по големина. Най-малкият брой сравнения $T_{\min} = n-1$ се получава при масив, сортиран в намаляващ ред, тогава се извършва само първата проверка, а най-много сравнения има ако $\min = a[0]$.

Основна тук е операцията сравнения и $\Rightarrow$ сложността спрямо нея е: $T(n)=n-1+n-1+p=2n-2+p \Rightarrow$

$$T_{\text{cp}} = 2n-2 - \ln_n$$



Примери за определяне на сложност на алгоритъм

Пр.2: Алг. за намиране на минималния (*min*) и максималния (*max*) елемент в 1-мерен масив $A[1..n]$ от реални елементи – 3 метода:

- 3 вариант- с 1 обхождане със стъпка 2:

23	4	18	7	35	22	8	15
За масива $\min=4$, а $\max = 35$							

```

main () {
  if (odd (n))
    {min=a[0];max=a[0];i=1;}
  else
    if (a[0]<a[1]) {min=a[0];max=a[1];i=2;}
    else {min=a[1];max=a[0];i=2;}
  while (i<=n)
    { if (a[i]<a[i+1])
      { minc=a[i]; maxc=a[i+1];}
      else {min =a[i+1]; max=a[i];}
      if (minc<min) min=minc;
      if (maxc>max) max=maxc;
      i=i+2;}
}

```

Приемаме, че n е четно, за 1-та двойка се прави 1 сравнение, и по-малкият ел-т се присвоява на *min*, а по-големия на *max*. За всяка следваща двойка се правят три сравнения. Min, max, и среден брой сравнения са равни ➡

$$\begin{aligned}
 T_{\min} &= T_{\max} = T_{cp} = 1 + (n-2/2) * 3 = 3/2 * 3 = \\
 &= 3/2 * n - 2
 \end{aligned}$$



Примери за определяне на сложност на алгоритъм

Пр.2: Сравнение на трите алгоритъма по сложност: (в брой операции сравнения при n-мерен масив)

Алгоритъм	Tmin	Tmax	Tcp.
1в. За 2 обхождания	$2n-3$	$2n-3$	$2n-3$
2в. За 1 обхождане, стъпка 1	$n-1$	$2n-2$	$2n-2-\ln n$
3в. За 1 обхождане, стъпка 2	$3/2n-2$	$3/2n-2$	$3/2n-2$

Средната сложност (брой сравнения) при различни ст-ти на n:

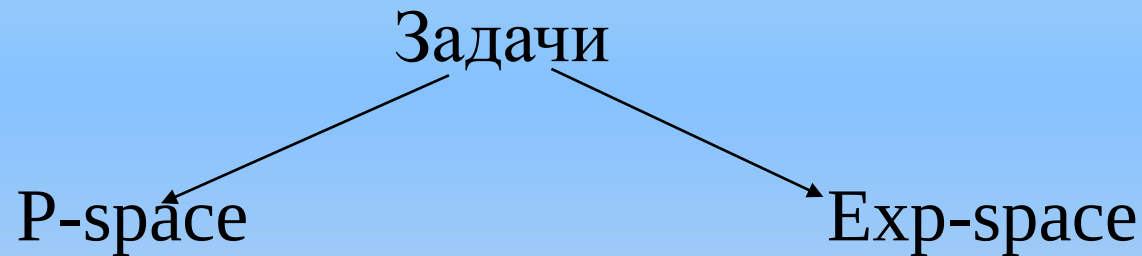
Алгоритъм	Tcp(n)	Tcp(100)	Tcp(1000)	Tcp(10000)
1в. За 2 обхождания	$2n-3$	997	1997	19997
2в. За 1 обхождане, стъпка 1	$2n-2-\ln n$	193,4	1991,1	19988,7
3в. За 1 обхождане, стъпка 2	$3/2n-2$	148	1498	14998

От табл. $\Rightarrow$ Най-бавен е алг. с 2 обхождания (1в.) и по трите критерия. Най-добра мин. сложност има алг. с 1 обхождане и 1 стъпка (2в.). Най-добра макс. и средна сложност има алг. с 1 обхождане и 2 стъпки (3в.)



Сложност на алгоритъм. Сложност по памет

(функция от големина на входните данни)



(задачи, за които е
необходима
полиномиална памет)

(изискват експоненциална
памет)



Сложност на алгоритъм. Сложност по памет

NP-задачи

(non-deterministic polynomial time
или полиномиално-проверими задачи)

Една задача принадлежи на класа **NP** , ако е възможно с полиномиална сложност да се провери, дали даден кандидат действително е решение, без да се интересуваме, колко време ще отнеме намирането на такъв кандидат.



Сложност на алгоритъм. Сложност по памет

Напр.: Дадено е естествено число n . Да се провери дали съществува делител на n , по-малък от дадено естествено число q :

$$q > 1, q < n$$

Ако разполагаме с кандидат за решение q' , лесно можем да проверим дали съществува p' , такова, че:

$$n = p' \times q'$$

Но за да решим формулираната задача, ще трябва по-големи изчислителни ресурси.



Сложност на алгоритъм. Сложност по памет

P-задачи

(polynomial) - задачи, за които съществува решение с полиномиална сложност)

Експоненциални задачи

- могат да се решат с експоненциална сложност



Сложност на алгоритъм. Сложност по памет

Нерешими задачи:

Не могат да бъдат решени независимо с колко време и памет разполагаме.

Например: да се реши уравнението:

$$a^n + b^n = c^n$$

a, b, c, n - естествени числа, $n > 2$

Съгласно скоро доказаната Голяма теорема на Ферма, такава четворка естествени числа не съществува.



Сложност на алгоритъм. Сложност по памет

Но програмистът (и компилатора) трудно може да прецени, дали решението съществува:

```
int main()
{
    unsigned a, b, c, x, n;
    for (x=3; ;)
    { for (a=1; a<=x; a++)
        for (b=1; b<=x; b++)
            for (c=1; c<=x; c++)
                for (n=3; n<=x; n++)
                    if (pow(a,n) + pow(b,n) == pow(c,n))      exit(0);
        x++;
    }
    return 0;
}
```




Сложност на алгоритъм. Сложност по памет

NP-пълни задачи (NP-complete)

$P \leftrightarrow NP$?

Т.е. ако проверката дали дадено кандидат-решение изисква полиномиално време, то и цялото решение също става за полиномиално време?

NP-пълни задачи е клас от NP-задачи, които могат да се свеждат една към друга с полиномиална сложност, и всяка задача от NP може да се сведе до някоя NP-пълна задача.

Ако само за една NP-пълна задача бъде доказано, че принадлежи или не принадлежи към класа P, то това ще следва и за всички останали задачи от класа NP.

(Например: търсене на прости пътища с дължина K в граф)



Сложност на алгоритъм. Обобщение

Най-често срещани стойности на $f(n)$ за $O(f(n))$

- 1
- $\log n$ $\log_2 n$ $\log_{10} n$
- n
- $n \cdot \log n$
- n^2
- n^3
- 2^n
- $n!$

Сложност на алгоритъм. Обобщение

Основни правила за определяне на $O(f(n))$

- Всеки прост оператор, който **не зависи от n** , има сложност **1**
- Два последователни програмни фрагмента с оценки на сложност $O(f_1(n))$ и $O(f_2(n))$ имат обща сложност **$O(\max(f_1(n), f_2(n)))$**
- Ако фрагмент със сложност $O(f_1(n))$ е вложен в друг със сложност $O(f_2(n))$, то общата им сложност е **$O(f_1(n) * f_2(n))$**
- Ако има **K** цикли, вложени един в друг, със сложност $O(n)$ всеки, общата им оценка е **$O(n^K)$** .